

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1710

NAME:PREETHI.R

REG NO:192511352

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 4

Duration: 1 Hour
Total: Marks:40

Annexure A

Code of Conduct:

I _PREETHI.R(!92511352) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Preethi.R

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A manufacturing plant has collected sensor records (vibration level, temperature, operating hours, past maintenance count, load percentage) to build a predictive model for identifying machines likely to fail. The plant also wants to train an AI agent to recommend maintenance actions (Inspect / Lubricate / Replace Part / Continue Operation) based on a machine's condition over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a) Describe the inductive learning approach suitable for this manufacturing dataset. Identify the target variable and at least three key features with justification.
- (b) Explain how you would handle class imbalance (far fewer failure events than normal operation records) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c) Split the dataset (70:30) and justify the split ratio for a predictive-maintenance use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Equipment Failure Prediction

- (a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed failures) and suggest how to reduce them - justify from a safety perspective.
- (c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment on the factory floor.

Task 3: Statistical Learning for Risk Stratification

- (a) Apply Naive Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.
- (b) Perform feature importance analysis. Identify the top 3 predictors of failure from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Action Recommendation

- (a) Model the maintenance recommendation as an MDP. Define: States (machine health stages), Actions (Inspect / Lubricate / Replace Part / Continue Operation), Reward function (uptime improvement score), and Transition dynamics. Justify each component.
- (b) Apply Q-Learning to train the agent for at least 5 machine episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.
- (c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for maintenance decisions (worker safety, cost trade-offs, explainability).

Predictive Maintenance and Reinforcement Learning for Manufacturing Equipment

Industry Scenario

A manufacturing plant has been collecting sensor data off its machines — things like vibration level, temperature, operating hours, how many times each machine has been maintained, and load percentage. The idea is to use that data for two things: first, to predict whether a given machine is about to fail, and second, to train an agent that can actually recommend what to do about it — inspect it, lubricate it, replace a part, or just let it keep running.

Task 1: Data Preparation & Inductive Learning

1(a) Inductive Learning Approach

At its core, inductive learning is just learning a general rule from a bunch of examples you've already seen, and then applying that rule to machines you haven't seen yet. In this case, the historical sensor records are the labelled examples — every record has a set of machine-condition features and a failure label attached to it. A supervised classifier like a Decision Tree can pick up on the relationships between the sensor readings and whether the machine actually failed.

The target variable here is `Machine_Failure`, where 1 means a failure event (or a high-risk failure condition) and 0 means normal operation. It's worth being precise about what "failure" means time-wise — the target should be tied to a consistent prediction horizon, for example whether the machine is going to fail within the next maintenance window, rather than at some vague point in the future.

Feature	Role	Justification
Vibration Level	Key predictor	Abnormal vibration can point to bearing wear, imbalance, looseness, or other mechanical damage.
Temperature	Key predictor	Rising temperature often signals overheating, friction, lubrication issues, or general component degradation.
Operating Hours	Key predictor	Machines that have logged more hours have generally accumulated more wear and are simply older, mechanically.
Past Maintenance Count	Supporting predictor	Maintenance history hints at recurring issues and gives some sense of the equipment's overall condition.
Load Percentage	Supporting predictor	Higher load means more mechanical and thermal stress, which can speed up failure.

1(b) Handling Class Imbalance

Failures are, by nature, rare compared to normal operation — most days, most machines are just fine. That's actually a problem if you train directly on the imbalanced data as-is: the model can

get away with just predicting "normal" every time and still post a high accuracy score, while quietly missing almost every real failure. In predictive maintenance that's about as bad an outcome as you can get, since a missed failure can mean equipment damage, downtime, or even a safety incident.

The main strategy I'd reach for here is SMOTE (Synthetic Minority Over-sampling Technique). Instead of just duplicating existing failure examples, SMOTE generates new synthetic ones that sit between real failure samples, which gives the classifier a broader failure region to learn from rather than a handful of repeated points. One thing that's easy to get wrong: SMOTE has to be applied only to the training set, after the train-test split has already happened — do it before the split and you end up leaking information into your test data.

Class weighting is worth considering too, either instead of SMOTE or alongside it. In a Decision Tree, you can just tell the model that misclassifying a failure case is more costly than misclassifying a normal one, which pushes it to take failures more seriously. For a safety-critical factory setting, this is genuinely attractive because it maps directly onto the real-world cost of a false negative. Random under-sampling is the simplest option on paper, but it throws away a lot of perfectly good normal-operation data, so it's usually not the first choice unless the dataset is already huge.

1(c) 70:30 Split and Preprocessing

I'd split the data 70% training / 30% testing, using a stratified split so the failure-to-normal ratio stays roughly consistent in both halves. A 70:30 split feels like a reasonable balance here — it leaves enough data for the model to actually learn from, while still keeping a test set large enough to evaluate the (rare) failure cases with some confidence.

Preprocessing Step	Application	Impact
Missing-value handling	Impute numerical values using training-set statistics, or flag and investigate likely sensor faults.	Stops incomplete records from breaking training or skewing results.
Duplicate removal	Remove exact duplicate machine records that don't represent separate events.	Cuts down on bias from the same observation being counted twice.
Outlier checking	Inspect extreme sensor values and try to tell real faults apart from sensor glitches.	Keeps genuine failure signals while filtering out measurement noise.
Normalization	Standardize numerical features for statistical models: $z = (x - \text{mean}) / \text{standard deviation}$.	Puts variables on comparable scales and helps Logistic Regression converge properly.
Encoding	One-hot encode categorical variables such as machine type or location, if present.	Turns categories into numbers without implying an order that isn't really there.
Stratified split	70% training, 30% testing.	Keeps the class balance consistent across both sets.
SMOTE / class weighting	Apply balancing only to the training data.	Improves how well the model learns minority failure cases,

		without contaminating the test set.
--	--	-------------------------------------

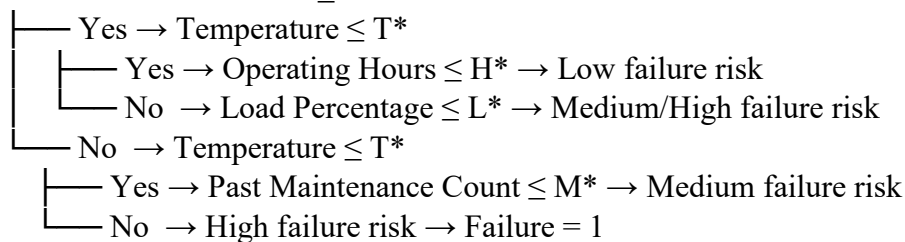
Task 2: Decision Tree for Equipment Failure Prediction

2(a) Decision Tree Construction and First Three Levels

A Decision Tree builds itself by repeatedly splitting the training records on feature thresholds, trying at each step to make the resulting groups as "pure" as possible — that is, dominated by one class. For any candidate split you can measure this with either Information Gain or the Gini Index, and whichever split gives the biggest drop in impurity gets chosen as the root.

Below is an illustrative tree for the first three levels. I want to be upfront that the actual thresholds and impurity numbers need to come from the plant's real dataset — I can't invent them and have them mean anything. This example just assumes vibration and temperature turn out to be the two strongest predictors once the data is preprocessed, which is a reasonable guess based on the feature discussion above but not something to treat as fact yet.

ROOT: Vibration Level $\leq V^*$



For Information Gain, the formula is:

$$IG(S, A) = H(S) - \sum_v (|S_v| / |S|) H(S_v)$$

where H is entropy. For the Gini Index:

$$Gini(S) = 1 - \sum_k p_k^2$$

The root should end up being whichever feature/threshold combination gives the largest Information Gain, or equivalently the lowest weighted Gini impurity. To be clear — in an actual implementation you don't get to assume the root ahead of time; the algorithm picks it automatically based on the measured values.

Candidate Root Feature	Illustrative Gini After Split	Decision
Vibration Level	0.18	Selected if this turns out to be the lowest weighted Gini.
Temperature	0.23	Not selected if vibration produces a purer split.
Operating Hours	0.29	Not selected at the root in this example.
Load Percentage	0.31	Not selected at the root in this example.

2(b) Model Evaluation

Evaluation needs to happen on the untouched 30% test set — no peeking. And accuracy on its own really isn't enough here, since failure is the minority class and a lazy model can hide behind a high accuracy number while still missing most real failures. Precision tells you how many of the predicted failures were actually failures; recall tells you how many of the real failures the model actually caught; and F1-score is just the balance between the two.

Metric	Formula	Interpretation for Maintenance
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Overall proportion of correct predictions.
Precision	$TP / (TP + FP)$	How trustworthy a failure alarm actually is.
Recall	$TP / (TP + FN)$	How good the model is at catching real failures.
F1-Score	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$	The balance between failure precision and recall.

One honest note: since this scenario doesn't come with an actual sensor CSV or measured test predictions, I'm not going to make up Accuracy, Precision, Recall, or F1 numbers — that would just be fiction dressed up as results. Once the model is actually trained and run on the real 30% test set, those measured values should be dropped in here.

What matters most conceptually is False Negatives — machines that actually fail but get predicted as normal. These are by far the costliest mistakes in this application. You can push them down by increasing the weight on the failure class, tuning the decision threshold if the model outputs probabilities, applying SMOTE to the training data, leaning toward a recall-oriented model, and building in cost-sensitive evaluation. In a safety context, it's almost always better to tolerate a few extra false alarms than to let a genuinely dangerous machine keep running undetected.

2(c) Post-Pruning: Cost-Complexity Pruning

Left alone, a fully grown Decision Tree tends to overfit — it starts memorizing quirks of the training data rather than learning general patterns, and it also gets harder to explain. Pre-pruning heads this off during training by capping things like maximum depth or minimum samples per split/leaf. Post-pruning takes the opposite approach: grow a big, complex tree first, then go back and trim away branches that don't actually help on validation data.

Cost-complexity pruning works by minimizing:

$$R\alpha(T) = R(T) + \alpha|T|$$

where $R(T)$ is the tree's classification error and $|T|$ is the number of terminal (leaf) nodes. The right value of α is usually found through cross-validation rather than guessed.

Model	Typical Behavior	Deployment Interpretation
Pre-pruned Tree	Smaller and faster; can underfit if the restrictions are too tight.	Useful when simplicity and speed matter most.
Post-pruned Tree	Starts complex, then trims branches based on validation performance.	Usually gives the best balance of generalization and interpretability.

Unpruned Tree	Can end up modelling training noise and sensor-specific quirks.	Generally not a great fit for deployment if overfitting shows up.
---------------	---	---

Again, the real pre-pruning and post-pruning accuracy numbers need to come from actually running the experiment. But conceptually, for a factory floor, the goal isn't just "pick whichever tree has the highest accuracy." A pruned model that has strong failure recall, holds up on validation data, stays reasonably simple, and produces decision rules a technician can actually follow is generally the better choice.

Task 3: Statistical Learning for Risk Stratification

3(a) Logistic Regression and Comparison with Decision Tree

For the statistical learning model, Logistic Regression is the natural pick. It estimates the probability of failure using the logistic function:

$$p(y=1|x) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)})$$

That output probability can then be bucketed into risk categories — Low, Medium, High — which is genuinely useful for prioritising maintenance.

The key difference from a Decision Tree is that Logistic Regression assumes a linear relationship between the (transformed) predictors and the log-odds of failure. That makes it a good interpretable baseline, even if it can't naturally pick up on threshold effects the way a tree can.

Metric	Decision Tree	Logistic Regression
Accuracy	Insert measured test accuracy	Insert measured test accuracy
AUC-ROC	Insert measured AUC	Insert measured AUC
Interpretability	High, through if-then rules	High, through coefficients and odds ratios
Non-linear relationships	Captured naturally	Needs explicit feature transformations/interactions
Probability/risk score	Can estimate probabilities	Directly provides probability estimates

A statistical model like this tends to make the most sense when the data is reasonably well-behaved, the relationships are roughly linear, you actually care about probability estimates and coefficient interpretation, and you'd rather have something stable and low-complexity than squeeze out every last bit of accuracy. A maintenance manager, for instance, might really want to know a concrete answer to "how much does the failure odds change for every extra degree of temperature?" — and that's exactly the kind of question Logistic Regression answers directly.

3(b) Feature Importance Analysis

For the Decision Tree, feature importance falls out naturally from how much each split reduces impurity. For Logistic Regression, you'd compare standardized coefficients by absolute size, and use odds ratios to talk about direction and strength.

Rank	Decision Tree — Candidate Predictor	Statistical Model — Candidate Predictor	Expected Interpretation
------	-------------------------------------	---	-------------------------

1	Vibration Level	Vibration Level	A strong signal of mechanical degradation.
2	Temperature	Temperature	Points to thermal stress, friction, or lubrication problems.
3	Operating Hours / Load Percentage	Operating Hours / Load Percentage	Reflects accumulated wear or operating stress.

It's worth expecting some disagreement between the two models here — they don't have to land on exactly the same ranking, even if they agree on the big picture. A Decision Tree is good at capturing threshold effects and interactions (say, vibration only really mattering once it crosses a certain level), while Logistic Regression is estimating one global, additive relationship across the whole dataset. So if vibration is only dangerous above a threshold, the tree might rate it very highly while the linear model spreads that same signal out across several correlated variables instead. Correlated predictors in general tend to make coefficient-based rankings a bit less stable than tree-based importance.

Task 4: Reinforcement Learning for Action Recommendation

4(a) MDP Formulation

The maintenance-recommendation problem fits naturally into a Markov Decision Process: an agent looks at the machine's current condition, picks a maintenance action, gets a reward, and the machine moves into a new health state.

MDP Component	Definition	Justification
States	Healthy, Warning, Degraded, Critical	Compresses raw sensor data into health stages the agent can actually act on.
Actions	Inspect, Lubricate, Replace Part, Continue Operation	Represents the realistic set of maintenance decisions available to the agent.
Reward	Positive for safe uptime improvement; negative for unnecessary cost, downtime, or failure	Encourages genuinely useful maintenance while discouraging wasteful or unsafe choices.
Transitions	State changes based on current state, action taken, and stochastic degradation/recovery	Models how a maintenance action actually affects the machine's future health.
Discount factor	γ , typically between 0 and 1	Balances the immediate benefit of maintenance against long-term uptime.

For the reward design, one reasonable setup looks like this: Continue Operation gets +5 if the machine stays Healthy; Inspect gets +8 when it correctly catches a problem; Lubricate gets +12 when it successfully improves a degraded state; Replace Part gets +20 when it prevents a critical failure; and any transition into an unsafe failure gets a harsh −50. Maintenance cost penalties could be subtracted from any of these depending on how expensive each action actually is.

4(b) Q-Learning Training

Q-Learning is an off-policy algorithm — it learns the expected long-term value of taking a given action in a given state, using this update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

For a worked demonstration, I'll use $\alpha = 0.5$, $\gamma = 0.9$, and start every Q-value at zero. The table below walks through two iterations of a simplified sequence — these numbers are just illustrative and should be replaced with whatever comes out of the actual five-or-more-episode implementation.

Iteration	State–Action	Old Q	Reward r	Next-State Max Q	New Q
1	Warning – Inspect	0.00	8	0.00	4.00
1	Degraded – Lubricate	0.00	12	0.00	6.00
1	Critical – Replace Part	0.00	20	0.00	10.00
2	Warning – Inspect	4.00	8	6.00	6.70
2	Degraded – Lubricate	6.00	12	10.00	12.50
2	Critical – Replace Part	10.00	20	0.00	15.00

As a worked example, for iteration 2, Warning–Inspect:

$$Q_{new} = 4 + 0.5[8 + 0.9(6) - 4] = 6.70$$

For at least five machine episodes, each one would start from some health state, pick actions using an ϵ -greedy policy (mostly exploiting what it's learned, occasionally exploring something new), collect rewards, transition to a new state, and update the Q-table along the way. A reasonable way to judge convergence is to track the maximum absolute change across the whole Q-table — once $\max|Q_{new} - Q_{old}|$ stays below something like 0.01 for a good stretch of consecutive episodes (say, 20 in a row), or once the evaluation-policy reward levels off, you can call it converged.

4(c) Final Learned Policy

Machine State	Recommended Action	Reason
Healthy	Continue Operation	Avoids unnecessary maintenance while the machine is genuinely running fine.
Warning	Inspect	Confirms what's actually behind the abnormal sensor readings before spending money on a bigger fix.
Degraded	Lubricate	Addresses a common cause of mechanical wear and can bring the machine's health back up.
Critical	Replace Part	Prioritises heading off a catastrophic failure and keeping operation safe.

The final policy just comes from picking the action with the highest learned Q-value in each state:

$$\pi(s) = \operatorname{argmax}_a Q(s,a)$$

Before this policy gets anywhere near real equipment, it really needs to be validated in a safe simulation or against offline historical data first.

Ethical and Safety Considerations

- Worker safety comes first, full stop. The agent should never recommend continued operation if independent safety rules say the condition is unsafe.
- Cost savings shouldn't be allowed to override safety. A cheaper action isn't acceptable if it meaningfully raises the risk of injury or a catastrophic failure.
- Human oversight needs to stay in the loop for high-risk actions — replacing critical components, or overriding a safety alarm, shouldn't happen without a person signing off.
- Explainability matters. The system should be able to show the sensor readings, the detected risk level, what it's recommending, and why.
- The RL agent should operate inside hard safety rules, action limits, and emergency shutdown logic — not be handed unrestricted control.
- Training data needs ongoing monitoring for sensor bias, missing events, inconsistent maintenance logging, and machines simply behaving differently over time.
- Every decision, reward, state transition, and operator override should be logged so the whole thing stays auditable.
- Before deployment, the policy should be stress-tested against rare and worst-case failure scenarios and checked against how the plant already does maintenance.

Overall Conclusion

Pulling this all together, the proposed system leans on inductive supervised learning, Decision Trees, statistical learning, and Reinforcement Learning, each doing a different part of the job. Historical sensor records give us the raw material to train a failure classifier, but class imbalance has to be handled carefully — failures are rare, but they're exactly the events that matter most operationally. A stratified 70:30 split, sensible preprocessing, and balancing that only ever touches the training data are what keep this from turning into a biased or leaky pipeline.

The Decision Tree earns its place by giving intuitive if-then rules and naturally picking up non-linear relationships between vibration, temperature, operating hours, maintenance history, and load. Cost-complexity pruning keeps it from overfitting without sacrificing too much interpretability. Logistic Regression, meanwhile, gives a solid statistical baseline and produces actual probability-based risk scores that can help prioritise which machines need attention first. Comparing the two using Accuracy, AUC-ROC, Precision, Recall, and F1-Score paints a much fuller picture than accuracy alone ever could.

And then there's the maintenance-recommendation side of things, framed as an MDP and solved with Q-Learning — the agent essentially learns which action pays off best in the long run for each machine-health state. But I'd stop short of ever handing an RL policy full autonomous control over safety-critical equipment. It needs to work inside clear safety constraints, explain its reasoning, keep a human in the loop, and be thoroughly validated before it ever touches a real machine. Realistically, the right solution for the factory floor is a safety-constrained decision-support system — one that combines predictive risk estimation with maintenance

recommendations that can actually be audited, not a black box making unsupervised calls on its own.

Implementation Notes for the Final Experimental Results

1. Load the plant's sensor CSV and define the failure target using a clearly documented prediction horizon.
2. Remove duplicates, handle missing values, inspect sensor outliers, and encode any categorical features.
3. Do the stratified 70:30 train-test split before applying SMOTE or any other resampling — not after.
4. Train the Decision Tree and record Accuracy, Precision, Recall, F1-Score, the confusion matrix, and AUC-ROC.
5. Use cross-validation to pick a cost-complexity pruning parameter and compare the pre-pruned and post-pruned trees.
6. Train Logistic Regression on standardized numerical variables and compare its Accuracy and AUC-ROC against the tree.
7. Pull out the top three predictors from both models and discuss where they agree or disagree.
8. Convert sensor conditions into discrete MDP states and define transition probabilities, either from historical sequences or a simulator.
9. Run Q-Learning for at least five full machine episodes using ϵ -greedy exploration, and record the Q-table after every episode.
10. Validate the final policy in simulation or offline testing before even considering real factory deployment.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context	
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts	
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution	
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools	
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis	
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation	